

# セッション再開ノート (最終更新: 2026-05-05)

2026-05-05

## Contents

|                                   |   |
|-----------------------------------|---|
| セッション再開ノート (最終更新: 2026-05-05)     | 1 |
| 1. プロジェクトの全体像                     | 1 |
| 2. 直近のセッションでやったこと (時系列)           | 1 |
| 3. 開発・実行の起動コマンド早見表                | 3 |
| 4. 直近の TODO / 次のセッションで再開する候補      | 4 |
| 5. リポジトリの主要な変更点 (このセッションで触ったファイル) | 4 |
| 6. 既知の挙動 / 制約                     | 5 |
| 7. 戻ってきた時の最初のチェックリスト              | 5 |

## セッション再開ノート (最終更新: 2026-05-05)

このメモは、対話セッションを一度終了して後で戻ってきた時に、最低限読めばどこまで進んでいるか分かるためのものです。

### 1. プロジェクトの全体像

ABCL/c+ は **3 ランタイム** を持つ actor 言語:

| ランタイム                | 場所                                                              | 役割                                              |
|----------------------|-----------------------------------------------------------------|-------------------------------------------------|
| OCaml                | src/*.ml (dune ビルド →<br>_build/default/src/repl_thread) (Cavea) | リファレンス実装、SDL/Web<br>Gateway 持ち                  |
| Python               | src/python-abcl/ (lark +<br>asyncio)                            | AI 連携の実験場、distributed<br>smoke、Docker container |
| JavaScript (browser) | src/browser-abcl/ (jison)                                       | Web デモ、可視化、教育用 UI                               |

すべて wire-compatible HTTP/JSON プロトコル (/api/json/send /api/json/call) で 相互通信できる。

### 2. 直近のセッションでやったこと (時系列)

#### 2.1 言語機能の追加

- ・ **now / future / await** を 3 ランタイムすべてに導入
  - OCaml: lexer/parser/AST/infer/eval\_thread.ml に追加
  - Python: grammar.lark + abcl\_parser.py
  - JS: grammar.jison + ast.js + runtime.js
- ・ **リモート now / future**: OCaml が HTTP /api/json/call で同期分散呼び出し
- ・ **session\_define (型付き) + protocol\_define (順序のみ)** を Python に実装 → src/python-abcl/abcl\_aios.py
- ・ **JS browser**: aios\_\* / protocol\_\* builtins を runtime.js に追加

- `aios_now` / `aios_future` / `aios_register_service` / `aios_emit` / `aios_services` / `aios_events`
- `protocol_define` / `protocol_start` / `protocol_state` / `protocol_end` / `protocol_events`
- `_dispatchAiosProtocol` で `CallExpr` と `CallStmt` 両方から共通 `dispatch`
- JS 文法に `bare IDENT(args);` の `CallStmt` 規則を追加 (`call` 不要に)

## 2.2 重要なバグ修正

- ・ OCaml の `current_msg_id` をスレッド・ローカル化 —parallel future で reply slot が混線していた race condition を修正 (Hashtbl.t keyed by Thread.id)
- ・ OCaml `value_of_json_atom` に `\uXXXX` JSON エスケープ復元 (UTF-8 エンコード)
  - サロゲートペア対応
- ・ Python container の locale: `LANG=C.UTF-8 PYTHONIOENCODING=utf-8` を渡さないと cold-start 時に Gemini レスポンスが cp1252 として decode される問題

## 2.3 Python 環境の更新 (案 2)

- ・ python3 を Python 3.13 に切替 (`brew install python@3.13`)
- ・ pyexpat の symbol mismatch を `brew install expat + install_name_tool -change + codesign --force --sign -` で修正
- ・ 依存 install: `python3.13 -m pip install --user --break-system-packages -r requirements.txt`
- ・ Makefile / `run_all_smoke_tests.sh` / `src/python-abcl/_smoke_test.sh` を 3.13 に切替済み

## 2.4 ブラウザ UI

- ・ `browser_console.html`: 右パネルに canvas + chat 吹き出しを併設、縦/横リサイズ
  - デフォルト幅 width: 66vw (= console 1/3 / right 2/3)
  - textarea height 260px (5 行増)
- ・ `cooperative_chat.html`: 専用吹き出しビュー (5 役: User/Planner/Solver/Reviewer/Verdict)
- ・ `cooperative_solve.html`: ワークスペース (console + chat + textarea + sample 切替)
  - 5 つのサンプル をドロップダウンで切替 (詳細は §3)
- ・ `rotate4lines_workers.html`: actor を Web Worker (真の OS スレッド) で動かす
- ・ Bounded Buffer: 15 スロット可視化 + Producer/Consumer 速度スライダー
- ・ Philosopher のフォーク矢印—canvas サイズに比例した正しい geometry に修正

## 2.5 Cooperative ワークスペースのサンプル一覧 (現在 5 種類)

| # | dropdown 値  | 名前                                    | 構造                                                        | 言語      |
|---|-------------|---------------------------------------|-----------------------------------------------------------|---------|
| 1 | cooperative | 標準<br>(now/future/await<br>+ AI mock) | Planner→Solver+brief<br>(Solver と brief 並<br>列)           | English |
| 2 | threeActor  | 3 アクター協調型<br>(aios_* +<br>protocol_*) | Planner が<br>aios_future で順<br>次                          | English |
| 3 | sequential  | 逐次分業型                                 | Coordinator が<br>aios_future +<br>await で 3 worker<br>を順次 | 日本語     |
| 4 | nowSend     | now メッセージ送<br>信型                      | Coordinator が<br>aios_now で 3<br>worker を順次               | 日本語     |

| # | dropdown 値 | 名前           | 構造                                                                                                               | 言語  |
|---|------------|--------------|------------------------------------------------------------------------------------------------------------------|-----|
| 5 | futureJoin | future 並列合流型 | MainThread が<br>now planner →<br>future solver<br>/ future brief<br>並列 → await 両<br>方 → now re-<br>viewer.review | 日本語 |

### 3. 開発・実行の起動コマンド早見表

#### 3.1 ブラウザを開く

```

bash /tmp/abcl_browser.sh
# port 8765 で http.server 起動 + ブラウザ自動オープン
# 主要 URL:
#   http://localhost:8765/                index
#   http://localhost:8765/browser_console.html  Console (汎用)
#   http://localhost:8765/cooperative_solve.html  協調ワークスペース (5 サンプル切替)
#   http://localhost:8765/cooperative_chat.html  吹き出し専用
#   http://localhost:8765/rotate4lines_workers.html  Web Worker 版

```

ハードリロード必須: **Cmd+Shift+R**

#### 3.2 OCaml ABCL/c+

```

cd /Users/kodamay/ocaml-app/abclcp-project
dune build
bash abcl/_smoke_test.sh # 52/52 pass
bash /tmp/coop_jp_ocaml.sh # AI 協調 (日本語、Gemini 実走)
bash /tmp/coop_remote_jp.sh # reviewer を Docker 別ノードで動かす

```

#### 3.3 Python ABCL/c+

```

cd /Users/kodamay/ocaml-app/abclcp-project/src/python-abcl
/opt/homebrew/bin/python3.13 abcl_main.py samples-ai/CooperativeNowFuture.abcl
/opt/homebrew/bin/python3.13 abcl_main.py samples-ai/SessionTyped.abcl # session 型チェッ

```

#### 3.4 全 smoke (3 ランタイム + Dist)

```

cd /Users/kodamay/ocaml-app/abclcp-project
bash run_all_smoke_tests.sh
# 期待: ABCL 52/52, JS syntax 7+ parse 4, Python 7/7, Dist 8/8

```

#### 3.5 puppeteer によるブラウザ動作確認

```

ABCL_CHROME='/Applications/Google Chrome.app/Contents/MacOS/Google Chrome' \
ABCL_BASE='http://localhost:8765' \
node /tmp/abcl_pptr_test/_check_*.mjs

```

## 4. 直近の TODO / 次のセッションで再開する候補

### 短期 (1 セッション以内)

- ・ JS 版 `wait(ms)` の改良 — 現在は `actor.__nextDelay` を立てるだけで メソッド body 内では実際には block しない。 `futureJoin` サンプルで 並列の wall-clock 効果が出ない原因。 `requestAnimationFrame` ベースで 可視化と組み合わせるか、 `busy-wait` に `limits` をかけるか。
- ・ JS 版に `session_* (typed)` を移植 — Python は実装済み、 JS browser には未着手
- ・ OCaml/JS 版 `protocol_* / aios_*` の追加 (現状 Python のみ完備)

### 中期

- ・ Cooperative-chat ページに「`session_state` を吹き出し横にミニ表示」 — `protocol/session` の進捗が右側パネルでリアルタイムに見える形
- ・ Cooperative-solve に“実 Gemini”モード — Node 側プロキシを立てて ブラウザから `sync XHR` or `fetch` で実 Gemini を使えるようにする (CORS 対策)
- ・ `session_define` を OCaml に移植 — `infer.ml` と独立した runtime checker として実装、 `abclc/ai-samples/` 用にサンプル追加

### 長期

- ・ Web Worker 版に `now/future/await` を入れる — `SharedArrayBuffer` + `Atomics.wait` で `reply slot` を `thread` 越しに同期
- ・ Bounded Buffer Worker 版 — `Producer / Consumer / Buffer` をそれぞれ 別 Worker に
- ・ `session` 型の静的検査 (Honda/Wadler 風 `dual` + `linearity`)

---

## 5. リポジトリの主要な変更点 (このセッションで触ったファイル)

### OCaml ランタイム

- ・ `src/lexer.mll` — `now / future / await` 追加
- ・ `src/ast.ml` — `Now / Future / Await` AST ノード
- ・ `src/parser.mly` — トークン + 式規則
- ・ `src/infer.ml` — `Now/Future/Await` を `TAny` で `permissive`
- ・ `src/eval_thread.ml` — `VFuture`, `reply slot` 機構, `now/future/await` の `eval`, `thread-local msg_id` (`_msgid_table`), `value_of_json_atom` で JSON 復元
- ・ `src/repl_thread.ml` — `reply primitive` で `in-process slot` を `fulfill`, `compile` に永続 `<top>` `actor`, `plain VarDecl/Assign` 評価
- ・ `src/remote_client.ml` — `remote_call` (`HTTP /api/json/call sync`), `find_substring / strip_http_headers / extract_reply_value`

### Python

- ・ `src/python-abcl/grammar.lark` — `await` キーワード追加
- ・ `src/python-abcl/abcl_parser.py` — `await_expr` → `CallExpr("await", [e])`
- ・ `src/python-abcl/abcl_interp.py` — `aios_/protocol_/session_*` `builtins`, `_b_await` の `auto-observe`, `_aios_dispatch` ヘルパ
- ・ `src/python-abcl/abcl_aios.py` — 新規モジュール (`aios` + `protocol` + `session`)
- ・ `src/python-abcl/samples-ai/CooperativeNowFuture.abcl` — 新規
- ・ `src/python-abcl/samples-ai/SessionTyped.abcl` — 新規 (型検査実演)
- ・ `src/python-abcl/samples-remote/reviewer_node_full_jp.abcl` — 新規

### JavaScript (browser)

- ・ `src/browser-abcl/src/parser/grammar.jison` — `now/future/await`, `bare IDENT(args); CallStmt`
- ・ `src/browser-abcl/src/parser/parser.js` — `jison` で再生成
- ・ `src/browser-abcl/src/parser/package.json` — 新規 (`"type": "commonjs"`)
- ・ `src/browser-abcl/package.json` — `"type": "module"` 追加

- ・ src/browser-abcl/src/ast.js —Now/Future/Await ノード
- ・ src/browser-abcl/src/runtime.js —reply slots、AIOS、Protocol、drainUntilSlot、buf\_\* prims、prod\_speed/cons\_speed、philosopher 描画修正
- ・ src/browser-abcl/src/interpreter.js —top-level env 共有
- ・ src/browser-abcl/src/ui/console\_browser.js —右パネル (canvas+chat)、リサイズ、speed-Bar 表示制御、Goto Chat / Goto Solve ボタン
- ・ src/browser-abcl/src/worker\_actor.mjs —新規 (Web Worker actor)
- ・ src/browser-abcl/src/worker\_runtime.js —新規 (Worker dispatcher)
- ・ src/browser-abcl/src/main.js —index ページ更新
- ・ src/browser-abcl/cooperative\_chat.html —新規 (吹き出し専用)
- ・ src/browser-abcl/cooperative\_solve.html —新規 (ワークスペース、5 サンプル切替)
- ・ src/browser-abcl/rotate4lines\_workers.html —新規 (Web Worker demo)
- ・ src/browser-abcl/run\_cooperative.mjs —新規 (Node 用ヘッドレスランナー)
- ・ src/browser-abcl/cooperative\_now\_future.abcl —新規 (Node 用サンプル)
- ・ src/browser-abcl/favicon.ico —404 抑制

## ビルド/環境

- ・ Makefile —PY ?= /opt/homebrew/bin/python3.13
- ・ run\_all\_smoke\_tests.sh —同
- ・ src/python-abcl/\_smoke\_test.sh —同
- ・ /tmp/abcl\_browser.sh —http.server 起動スクリプト
- ・ /tmp/coop\_jp\_ocaml.sh —OCaml AI 協調起動
- ・ /tmp/coop\_remote\_jp.sh —OCaml + Docker reviewer 起動

## 6. 既知の挙動 / 制約

- ・ **JS browser での wait(ms)** は actor の次メッセージ待機を立てるだけ。メソッド body 内では block しない。futureJoin の“並列で 2 秒”は ブラウザでは即時完了する。
- ・ **OCaml の Sys.command** は thread 非安全。並列 future + Gemini 実走時は ABCL\_AI\_MAX\_CONCURRENT=1 を設定して直列化する必要がある。
- ・ **ブラウザ AI:** CORS の都合で実 Gemini は browser から直接呼べない。mock のみ。実 LLM が必要な時は Node 版 (run\_cooperative.mjs) または OCaml/Python 版を使う。
- ・ **Python container locale:** docker run 時に -e LANG=C.UTF-8 -e PYTHONIOENCODING=utf-8 を渡さないと、cold-start 時の Gemini レスポンスが cp1252 になる。

## 7. 戻ってきた時の最初のチェックリスト

```
# 1. プロジェクトに入る
cd /Users/kodamay/ocaml-app/abclcp-project

# 2. ビルドが通るか
dune build && echo "OCaml build OK"

# 3. smoke 全部通るか
bash run_all_smoke_tests.sh
# 期待: All smoke tests passed.

# 4. ブラウザを起動して触る
bash /tmp/abcl_browser.sh
# → http://localhost:8765/cooperative_solve.html でドロップダウン 5 種類
```

何かおかしかったら、まず git status で何が変わっているか確認、それから git log -10 --oneline で直近のコミット履歴を見る。